1.2 Fundamentos B

Funciones

A medida que tus programas crecen, te darás cuenta de que repites las mismas secuencias de código una y otra vez. Las funciones son la solución a este problema. Una función es un bloque de código reutilizable que realiza una tarea específica.

Definir funciones te permite:

- **Organizar el código**: Dividir un programa grande en partes más pequeñas y manejables.
- **Reutilizar código**: Escribir una lógica una vez y llamarla tantas veces como necesites.
- **Mejorar la legibilidad**: Darle un nombre descriptivo a un conjunto de operaciones hace que tu código sea más fácil de entender.

La sintaxis para definir una función en Python es simple. Se utiliza la palabra clave *def*, seguida de un nombre para la función, paréntesis () y dos puntos :. El código que pertenece a la función debe estar indentado.

Sintaxis

Python

```python
def nombre_de_la_funcion():
    # Código que ejecuta la función
    print("¡Hola desde una función!")
```

Para usar o "llamar" a la función, simplemente escribe su nombre seguido de los paréntesis.

Python

```python
# Llamando a la función
nombre_de_la_funcion()
```

Parámetros y Argumentos

Las funciones se vuelven mucho más poderosas cuando pueden recibir información para trabajar. Los datos que una función recibe se conocen como parámetros (cuando defines la función) y argumentos (cuando la llamas).

Python

```python
# 'nombre' es un parámetro
def saludar(nombre):
    print(f"¡Hola, {nombre}! Bienvenido.")

# "Ana" y "Carlos" son argumentos
saludar("Ana")
saludar("Carlos")
```

Profundicemos un poco en el tema de los argumentos que es importante e interesante

Puedes definir múltiples parámetros, separándolos por comas.

Python

```python
def sumar(a, b):
    resultado = a + b
    print(f"{a} + {b} = {resultado}")

sumar(5, 3)
sumar(100, -20)
```

Si queremos que la función reciba una cantidad indefinida de argumentos, puedes usar **args*:

Python Copiar

```python
def mostrar_nombres(*nombres):
    for nombre in nombres:
        print(nombre)
```

- Aquí, nombres es una tupla (similar a un array en JS).
- Puedes llamar *mostrar_nombres("Ana", "Luis", "Carlos")* y se imprimen todos.

Quiero que notes por favor que si imprimo nombres me va a regresar un objeto iterable de tipo *tupla* con todos los elementos que le pase a la función *mostrar_nombres,* asi mismo,

mucha atencion con esto, si pasas una lista(array) a una función que recibe *args entonces tendras algo asi al observar args ([foo, bar,xoo],), porque como sea va a ser una tupla.

Para recibir argumentos con nombre (como objetos en JS), usas ***kwargs*:

```python
def mostrar_info(**info):
    for clave, valor in info.items():
        print(f"{clave}: {valor}")
```

- *info* es un diccionario.
- Puedes llamar *mostrar_info(nombre="Ana", edad=30)* y accedes a los datos por clave.

Para usarlo por ejemplo

```python
mostrar_info(nombre="Ana", edad=30, ciudad="Caracas", ocupacion="Ingeniera")
```

Al ejecutarlo obtendremos

```
Nombre: Ana
Edad: 30
Ciudad: Caracas
Ocupacion: Ingeniera
```

Ojito que para poder pasar un diccionario como variable a una función, esperando que ***kwargs* haga su trabajo entonces tenemos que hacerlo de la siguiente manera

Python Copiar

```python
def ejemplo(**kwargs):
    print(kwargs)

datos = {"nombre": "Ana", "edad": 30}
ejemplo(**datos)
```

Es decir, que cuando llames la función *ejemplo* entonces tienes que colocar los ** antes que el nombre del diccionario, si no lanzara un error de tipo.

El Valor de Retorno o *return*

Muchas veces, no solo quieres que una función haga algo, sino que también te devuelva un resultado para que puedas usarlo más adelante en tu código. Para esto se utiliza la palabra clave *return*.

Cuando Python encuentra una sentencia *return*, sale inmediatamente de la función y envía el valor de vuelta.

Python

```python
def multiplicar(a, b):
    # La función calcula el producto y lo devuelve
    return a * b

# Guardamos el valor devuelto en una variable
resultado_multiplicacion = multiplicar(7, 4)

print(f"El resultado es: {resultado_multiplicacion}")
print(f"Podemos usarlo en otras operaciones: {resultado_multiplicacion + 10}")
```

Una función puede no tener una sentencia *return*. En ese caso, implícitamente devuelve *None*.

Argumentos por Defecto

Puedes asignar un valor por defecto a un parámetro. Si al llamar a la función no se proporciona un argumento para ese parámetro, se usará el valor por defecto.

Python

```python
def crear_personaje(nombre, clase="Guerrero"):
    print(f"Personaje Creado: {nombre}, Clase: {clase}")

# No especificamos la clase, así que usa el valor por defecto
crear_personaje("Aragorn")

# Aquí sí especificamos una clase diferente
crear_personaje("Gandalf", "Mago")
```

Funciones lambda

Una función lambda es una forma rápida de definir una función sin nombre, que contiene una sola expresión. Se usa comúnmente como argumento en funciones como *map(), filter(), sorted(),* entre otras.

Python Copiar

```python
lambda argumentos: expresión
```

Ejemplos

Python Copiar

```python
doble = lambda x: x * 2
print(doble(5))  # Salida: 10
```

Python Copiar

```python
suma = lambda a, b: a + b
print(suma(3, 7))  # Salida: 10
```

Python Copiar

```python
numeros = [1, 2, 3, 4]
doblados = list(map(lambda x: x * 2, numeros))
print(doblados)  # [2, 4, 6, 8]
```

Python Copiar

```python
numeros = [-2, 0, 3, 5]
positivos = list(filter(lambda x: x > 0, numeros))
print(positivos)  # [3, 5]
```

Para terminar con el tema de las funciones y dejar que el cerebro se relaje un poco vamos a terminar explicando un último punto

Funciones expresadas y declaradas

Las funciones declaradas son las funciones tradicionales que se definen con la palabra clave *def* y tienen un nombre:

Python Copiar

```python
def saludar(nombre):
    print(f"Hola, {nombre}")
```

- Se definen en el flujo principal del código.
- Están disponibles desde el punto en que se definen hacia abajo.
- No se pueden usar antes de su definición

Las funciones expresadas son funciones que puedes **asignar una función a una variable**

Python Copiar

```python
def despedir(nombre):
    print(f"Adiós, {nombre}")

mi_funcion = despedir
mi_funcion("Ana")  # Adiós, Ana
```

- Aquí, *despedir* es una función declarada, pero *mi_funcion* es una expresión que apunta a una función.
- También puedes definir funciones dentro de otras funciones o estructuras, lo que las hace más dinámicas.

¿Cuando se usan?

- Usa *def* cuando necesitas una función completa, reutilizable y clara.
- Usa lambda o expresiones cuando necesitas una función rápida y simple, como en *map(), filter(), o sorted().*

Ver archivo Funciones.py